H. G. Kolsky

COMPANY CONFIDENTIAL 5/4/59

Introduction to the STRETCH System

A talk by F. P. Brooks, Jr.

(Transcribed from a presentation given by F. P. Brooks)

1. Objectives

When the STRETCH project began, two important objectives were set for the resulting machine. The first is a performance objective. When the system was first proposed in 1955, the overall performance goal was set at 100 times the 704 speed, with the improved capacity which this improved speed implies. Some investigations indicate that a speed improvement of 100 times demands about 10 times memory capacity.

The reasons for wanting an improved performance are twofold. One is to permit problems to be solved which can't be solved with lesser machines. The second is to improve the efficiency on smaller problems, because this machine is still on the part of the cost-performance curve where a faster machine means less cost per unit of operation.

In an early article, which appeared without benefit of IBM help, Ward Sangren estimated the rental of our machines and the rental of competitive machines, the speed of our machines and the speed of competitive machines. He estimated that the STRETCH machine would have a per-operation cost about one fifth that of its next competitor, the LARK. Since higher speed means lower cost per operation, for a fully occupied machine, one can indeed improve the efficiency of solving smaller problems -- and this is a basic economic justification for building a high performance machine. So these are the reasons for the high-performance objective.

The initial objective is generality of applicability. This machine should be general purpose in a sense no IBM machine has ever been: so general that it would serve for the presently obvious applications in the field of science; so general that it would serve as our top machine for the conventional commercial application -- the tape spinning application; so general that it could do the other data processing jobs which are not conventional but which require extensive manipulation on great quantities of data. The Harvest attachment to this machine is one which is designed for special data processing jobs. This attachment should be handy on jobs such as weather, data reduction, or other unconventional but large scale data processing jobs.

Finally, the STRETCH system should be useful for integrated or real-time systems. Now, of course, any business system is a real-time system. It may be a slow-scale real-time system, in that one may not need a report but once a month; but it is still real-time, and if the first of the month comes and the report isn't there, the machine has failed in its task. This failure is just as definite as if the machine were guiding a missile and it didn't get the signal out at the time the missile needed it.

But when we say real-time, we generally mean fast real-time. There is a definite need for machines which can participate in fast real-time systems, machines which can be integrated with other automatic data handling devices so that the slow and error-prone human link is out of the system. This system had to be designed to fill this need, too.

Well, should the system be designed for so general a purpose? Why not build a lot of different ultra-high performance systems in different areas? One reason is that the cost of developing and building each different machine of this performance is just plain fantastic.

Another more basic reason is that the cost of the machine is largely tied up in the components which cannot be specialized. The greatest contributor to the cost of most STRETCH systems will be the memory boxes. All kinds of systems need memory boxes. In the main frame, to get the ultra-high performance needed, we have an index mechanism. Most problem areas need indexing in one form or another. We have an instruction look-ahead. Almost all ultra-high performance systems need something of this kind to use to use instructions rapidly.

A third reason arises from the fact that the number of machines one expects to sell is somewhat smaller than for the 650. This thinner market means that the development cost constitutes a higher proportion of the cost of each unit than it does for a smaller machine. And this economic fact strongly urges one to develop no more different machines than necessary. For these reasons, we have felt that in one development effort we must put, if necessary, just a little bit of extra thought and equipment here and there to make the machine applicable economically and conveniently to the highest performance needs of all of the wide range of problem areas.

A final reason is that the needs of even a single customer can rarely be compartmentalized. Most computer installations have a problem mix that includes almost all kinds of calculations.

These then were the objectives: very high performance - for solution of problems otherwise insoluble and for more economical solution of problems than otherwise possible; and generality of purpose - so that a single very high performance machine could fit the problem areas we can now see or anticipate.

2. Technology

The STRETCH program, however, did not spring into life from the existence of a need. New computers are born as much from new technology as they are from new problems. The thing which made the STRETCH machine a timely proposition was the development of a new transistor technology. New drift transistors in new current mode circuits promised circuit speed much faster than anything heretofore possible, and, just as important, promised component reliability so much higher than anything earlier that we could consider putting a large number together in a system. One just doesn't build a computer with 250,000 vacuum tubes. But the drift transistor seems to promise such that a machine of such size can be made to operate reliably.

Coupled with the advance in transistor technology, there came an advance in core memory technology. It was believed at the time the program was started that by pushing the new 30 - 50 mil cores to the very limit, we could build a memory whose cycle time was not 12 microseconds, as is the case in the 704, nor nine microseconds, as is the case in the 705, nor 6 microseconds as is the case in the SAGE machine, but two microseconds. This belief was substantiated. The cycle time at present, I think, is down to 2.2 microseconds, and development is not complete. The advances in transistor and memory technology were coupled with many new advances in packaging techniques. Printed circuits, low voltage connectors, frames which made for greater accessibility, cable improvements, and other such advances made it seem feasible for the company to undertake a program to explore this new technology. Steve Dunwell and other original planners of STRETCH decided that the most effective way to explore a potentially high performance technology was to use it in a very high performance computer which

would press the technology to the utmost. The wisdom of this decision has been demonstrated by the ease with which the STRETCH-pushed technology has been turned to the development of the 7090 and other 7000 series computers.

3. Power from Machine Organization

However, technological improvement is not enough. If one builds a memory six times as fast as previous memory and devises circuits which are 20-50 times faster than previous circuits, the combination does not constitute a machine which is a hundred times as fast as earlier machines. Therefore, if one hundred times is the performance goal, there must be radical changes in the system organization as well.

Harwood Kolsky has outlined the nature of some of the new organization by which the system does more things at once. Several kinds of overlapping are used. The exchange operates independently of the computer, and the several channels in the exchange operate simultaneously and independently of each other. Thus, up to 32 input-output operations can be proceeding at once, while a computation is going on as well. Inside the main frame, one could have further simultaneity of computation in the sense of two problems actually being calculated at once. This approach is embodied in the Bull Gamma 60 machine. Instead of this approach, we try to be working on several parts of the same problem at any instant through the use of the look-ahead. The arithmetic unit may be working on one part of the problem, while the index arithmetic unit is several instructions ahead on the same problem getting things ready, and while the look-ahead is fetching data for still another instruction. This is a different kind of parallelism in which, instead of attacking multiple problems at once, the machine tackles the same problem

at several different places.

The system organization is improved in other respects, as well. The provision of a more efficient instruction set makes considerable contributions to the performance of the computer. In most problems, a program requires about 20% fewer instructions to solve a given problem than it would on the 704 or 709. Thus, there are fewer instructions executed, fewer instructions stored, fewer instructions written by the programmer -- all of which improve performance and economy.

Not only was the technology inadequate for the performance improvement expected, but an extrapolation of present organization was inadequate for the generality objective. A simple extrapolation of 7090 with one more set of frills could not make a machine sufficiently applicable to commercial work, and real-time and control problems. The conceptual framework first built into the 701 just didn't extrapolate another stage to give the generality needed.

So for this reason, the organization had to be new in many respects, and it is new, but not radical. We haven't discarded the basic concepts of operation and address, of programs composed of sequences of simple instructions, etc. I think it will be some time before we fully outgrow these concepts. The newness of the organization has several aspects. First, there are a lot of what one might call features. These are not the result of a "Cadillac" complex, a logic which says that in the most powerful computer, the ashtrays should be gilded and the carpets should be plush. On the contrary, we believe that each one of these can be justified on the basis of an actual performance gain due to more efficient use of memory. Since the performance gain is most limited by the fact that the memory is only six times faster, we have had to strive for performance improvement by making better use of bits which come from memory. Consequently, an instruction set which takes ful advantage of the information that

can be packed into an instruction word is a necessity for a high performance machine with such a memory.

The second reason for many features is that we are now realizing that the programming cost of a machine is a real cost to the user, just as the rental is. And we believe that the provision of these features will make the programming easier.

These features will not make the machine easier to learn initially. The LGP-30 is the easiest machine to learn that I know of. It is not an easy machine on which to do a complex computation. Likewise, the 602A is an easy machine to learn; but arranging for those selectors to drop out at the right time is painful when one wants to do something difficult. Thus, when I speak of programming ease, I don't mean that a load - add - store loop is easier on this machine than it is on the 650 or 704. I do mean that the complex computation is easier -- the one which ordinarily makes the programmer think of a half dozen devious schemes and then spend still longer deciding which one has the greatest aesthetic appeal.

These difficult tasks we have attempted to make straightforward without knowing exactly what they are going to be. We cannot know what people will want to do with this machine -- we cannot foresee all the specific needs, but we can foretell the general kinds of complex operations. So we have attempted to provide general purpose machinery for simplifying many of the involved programming tasks. Throughout the weeks to come we will try to go into the features one at a time and show how it meets a general need and show a few examples of what its application might be.

A second aspect of the organization is the emphasis upon unity of concept. Provision of numerous features often means a terrifying increase in the conceptual complexity of a system -- the things one has to learn. This we have attempted to avoid by arranging the instruction set systematically, by introducing symmetry where possible, by solving similar problems in similar ways in different parts of the machine. As an example, consider the matter of symmetry in the instruction set. The sign modifiers mean that one has absolute value modifiers on all floating point commands, instead of having specific absolute value commands. The specific need for an add and a subtract is met in a more general way with a sign modifier on all arithmetic commands. If one wants to change the sign of a word as it goes to memory, one can specify store with invert sign, leaving the accumulator the way it was. Symmetry works another way: where possible, the floating point arithmetic and the integer arithmetic have the same instructions, and, where possible, the same instructions occur in the index arithmetic set.

The fact that things are general and systematic means that there is less to be learned. The combinations of things one can do with a single instruction are approximately ten times as many as in the 709 with its 200 odd instructions. And yet we have only 80-odd operation codes or some 40% as many as the 709 has.

This reduction is possible only because we were not constrained to build on the framework of the 701, as the 709 designers had to do. We could incorporate the needed instructions with a systematic arrangement from the beginning. This freedom from the shackles of history has been no small gift to the machine (and to its designers).

Now, the manual which you have is a little frightening in its thickness, and, as you dip into it, in its density. I think this is not truly representative of the relative complexity of this machine. One thinks of the 650 Preliminary Programming Manual as a paper-bound book and compares it in density and thickness with this, and one thinks: "This must be the most awful thing that has ever come along". Well, one is indeed awe-struct upon contemplating all the hardware and all the things it must do. I think, however, that the relative thickness of the manual is due in good measure to the fact that we are farther along than most preliminary manuals. Most of the loose ends have been run down and put into writing -- which I think is not true in most preliminary manuals. Thus, many exception cases and contingencies are covered in the text and numerous programming notes explain these.

So, don't be frightened by the manual. It is intended not as a pedagogical manual, but as a Reference Manual -- a book in which you can find out, for example, everything we know about floating point add. Of course, this description is not necessarily all in the same place, in the manual in spite of our attempts.

For a pedagogical manual we will have to rely on the Commercial Development people and the others who take up the load from here. We felt that the first need, because this is the manual from which the machine is being built, was a manual as detailed as we could write, a manual in which everything known is written down. We believe that this is also what the programmer needs for his reference purposes. He, too, wants to know how the odd-ball case is going to work.

A third aspect of the new organization is that we have made every attempt to avoid specialization at the expense of generality. This is based upon the assumption that we cannot know how the customer will want to use this machine. We are working with a machine whose potential users are sophisticated and supplied with very difficult problems. I think that no amount of planning can foresee the needs which will come up in the lifetime of a machine like this, and I think that anyone who claims that he can fully foresee these needs is fooling himself.

Consequently, we felt that it would be a mistake to isolate a need which is foreseen and put in a little gimmick that did that job perfectly but missed a family of related needs. So, we tried to provide general purpose data manipulating tools. Here are a few examples.

Our editing facilities are not so slick as the 709's convert operations for the three jobs for which the convert operations were designed. STRETCH facilities will, however, accomplish cleanly a very wide range of operations, and they will do those three with respectable smoothness. I hope this will be demonstrated before the week is out. As a general purpose editing machine, STRETCH has quite a bit of power. The logical operations we have provided are not confined to the things that we could demonstrate a need for. As a matter of fact when we started we put in all the sixteen logical connectives only because of our general principle of symmetry. We knew that nobody really needed any more than and, or, not and exclusive or. But we threw the rest of them in. Lo and behold, in our programming we find that the ones used most often are the six trivial ones, and that the four which we would have provided if we had restricted ourselves are much less used than these six.

Similarly, in the interruption system design, we have avoided the type of trapping that forces the program to a fixed location or performs a hardware-specified fix-up function. On the contrary, the programmer is given control as to the conditions that cause interruption, the location of the interruption instructions, the location where the instruction is stored, and whether interruption is permitted at all or not.

The avoidance of specialization that penalizes generality has been carried to the extreme in the design of the console. The console is liberally provided with many kinds of keys, switches, lights, and displays. There is even a click and a chime. Lest we wrongly decide how to define the function of any button, we defined them all so that their actuation does nothing at all. The switches set up a pattern of bits which the machine reads as if it were a tape or a card reader. These are interpreted by a console-defining routine to give whatever function is desired. A pattern of bits written out to the console sets up the lights and displays as desired.

Where specialization of function is demanded for machine performance, we have not hesitated to provide it, but in addition to generalized methods of accomplishing the same ends. Indexing and floating point arithmetic are prime examples of specialized features which are logically redundant but which contribute immensely to system performance. There are many smaller examples.

4. Provision for New Methods of Operation

A factor of importance in the planning of STRETCH has been the need to provide for new methods of operation. The need is certain; a STRETCH computer is by no means a magnified and accelerated 704, and will not be used in the way a 704 is used. A performance ratio of 100 times is itself sufficient

to make two computers differ in kind rather than degree. The many extra facilities of STRETCH further accentuate this difference.

Therefore, it has seemed wise to attempt to think through the ways in which one would like to use this machine, and to provide the hardware additions necessary to facilitate the desirable new modes of use.

For example, one new mode of use is in real-time operations. This immediately suggests the need for special emphasis on reliability and service-ability, and this emphasis has been great.

Among other new use techniques is multiprogramming, which I think deserves some special discussion. The complexities of multiprogramming are very great, but the urge toward multiprogramming is almost irresistable. It arises from some very basic economic considerations.

For many reasons, the performance of a computer increases more rapidly than linearly as cost increases, at least up to a very high point. This means that the job cost on a fully loaded powerful computer is lower than it is on a less powerful machine. Therefore, as the work to be done increases, there is a very strong economic incentive to do it with faster computers rather than more computers. This is a basic reason for the development of more powerful computers.

Now, as the power of a computer increases, the cost also generally increases. Thus, while an addition is cheaper on a faster computer, a second's wait is more expensive. This means that the desirability of avoiding the waste of seconds increases as speed increases. The design of a high performance machine must show recognition of this economic pressure.

Where do wasted seconds occur? The most obvious place is in the input-output area. Such a waste occurs whenever the machine asks the external world for a reaction from a control system or human being. A machine stop for operator action is the extreme case. Because these seconds are necessarily very costly in a high performance machine, we must figure out a method of avoiding their waste.

However, there are some waits in an individual program which cannot be economically avoided. If this be true, the only way to avoid wasted seconds is multiprogramming; i. e., having available for the high performance mechanism a number of tasks, so that the necessary time waits in each task can be utilized by operations on one of the other tasks.

In 1956 this wasn't obvious, and we proceeded ahead with the multiprogramming philosophy on faith rather than on certainty. I think now our certainty has increased. Similarly, the Honeywell 800 advertising makes it obvious that its designers are banking heavily on multiprogramming. The Bull Gamma 60 is organized around the basic assumption of multiprogramming. I do not believe these competitors are mistaken; I think that the economic case for multiprogramming is indisputable. What then are the requirements for satisfactory multiprogramming?

The most obvious requirement is an interruption system. No new machine has appeared on the market - or rather, no new medium or large machine has been announced, in the last 12 months without an interruption system of some kind. An interruption system is not only quite useful for programming and multiprogramming, but it is also necessary for efficient operation of an independent input-output system, such as the exchange or data synchronizers.

We believe that there is another hardware device which is of equal importance for multiprogramming. This is address protection, which permits areas of memory to be protected from change. If one has several jobs on a machine, each one can be protected from damage by straying of the others. This is essential for multiprogramming, because in the real world programs do go astray and in the real world people don't put their programs on a machine with other people's programs if they suspect that the other programs may interfere with their own. Most people would prefer inefficient machine operation to the headaches of crosstalk between buggy programs.

For these reasons we think that an address protection mechanism is just as essential to multiprogramming as is an interruption system. The Honeywell 800 people apparently think so, too. They block the memory off into 8 blocks and make it difficult to get from one to another. This provides a form of address protection. When one is multiprogramming with eight instruction counters, nothing makes better sense provided you happen to have eight jobs, each of which requires one block of memory. In addition to this mechanical address protection, they also put in hardware that changes the control instruction counter, etc., automatically when program changes are timely.

We believe that a programmed approach to multiprogramming is better than a hardware approach. The 800 advertisement described any programmed approach as "tedious and complex programming". It is tedious and complex. It is a difficult programming task -- the most difficult I have ever seen. It is a task, however, which does not have to be done but once. When we have finished developing a general multiprogramming system for STRETCH, it should not have to be done again by the customers or ourselves.

Why use a programmed approach instead of the hardware approach? The answer in a word is flexibility. The reason we put plugboards on our punched card machines is because plugboards permit the user to alter the allocation of all the facilities of the machine. This flexibility is very valuable. Similarly, in a multiprogrammed computer, one wants the ability to allocate all the facilities freely, so that if one program needs all the memory, it can have it. In otherwords, this is another reflection of the fact that we do not know what the programmers will need to do, and we cannot presume to know. Only by giving the programmer himself the ability to control these facilities, and necessarily, the task of controlling them, can we give him the full use of the powers of the machine.

Another basic philosophy reflected in our preference for a programmed approach to multiprogramming is that when the choice must be made, we prefer to give the user power rather than protection. Consequently, we have made every attempt to give the programmer access to all the hardware facilities, even those whose use is subtle and intricate.

Now this philosophy does not rule out protective features, for these are provided in abundance where they do not interfere with utilization of the machine. Undefined addresses and operation codes actuate special indicators, which can give interruption. Other indicators warn of floating point anomalies, loss of precision, overflows, attempts to read previously protected addresses, etc. When such peculiar operations are desired, however, they may be allowed to proceed without interruption. We have not excluded the oddball operation in order to protect the programmer. Essentially, the assumption is that the users are adults at least as smart as we, not children who must be protected from themselves. This philosophy has been challenged, but we believe it is the only

proper attitude with which to approach the design of a high-power computing machine.

Now, what will be the consequences of multiprogramming upon operating techniques? The fact that we avoid machine waiting when a program waits means that the cost of suspending a program until a human reacts is only the cost of the man's seconds and not the cost of the machine seconds. This is a radical change in the economics of system use. The natural and inevitable consequences of this economic change will be that we will begin to integrate people more closely into our mechanized problem-solving processes. I think that many of the very peculiar exception cases whose treatment now bulks so large in commercial codes will instead come to be decided by human judgement while the program waits. I think that in scientific computation, we shall see the problem sponsor, the man who knows the physics of the problem, come into communication with the machine so that he can help the machine find its solution and so that he can really learn what is happening down in the process of solution. Only in this way can he get the insights that the page of printed final results cannot convey. I think we will begin to see problem-solving systems which originally were exclusively men or exclusively machines increasingly become man-machine complexes.

5. The System

So much for philosophy. In the time that remains let us look quickly at the machine and its features. The system diagrams which were handed out to you include a simple one and a less simple one. Let's talk about the simple one. At the top of the diagram we see the keystone of any computer -- the memory. In this machine the memory comes in boxes of 2^{14} words or 16,384 words. Each word contains 2^6 or 64 bits. With enough money, one can get

2^4 or 16 boxes of memory. If one gets six or more boxes, memory references can be in process in up to six of these at a time. The addressing system provides for 2^{18} sixty-four bit words or 2^{24} bits, about seventeen million bits. That is a lot of information. The memory boxes operate at a cycle of about two microseconds. Because of the ability to have multiple references going at once, it is possible to use from two to four times as many words per second as the cycle time implies. The memory system is equipped with automatic correction for all single errors and indication for double errors.

There are in the machine three arithmetic units. Each is used by instructions that incorporate many new concepts. One unit does arithmetic on floating point data. This is one of the first modern computing machines to be designed around a floating point number system, instead of having a floating point arithmetic attached as an afterthought. The second arithmetic unit operates serially with either the binary or decimal radix upon variable length fields. The third arithmetic unit operates in parallel upon index fields and addresses, operating with the binary radix upon fields of fixed length and orientation. The index arithmetic unit does not provide multiplication or division.

The indexing system is built around the index adder and sixteen index registers which are realized by a separate small core memory. In addition to the type of indexing which is similar to that in the 704, there is a second mode called <u>progressive indexing.</u> Indexing is used not only for scanning arrays, but also for relative addressing. The progressive mode of indexing is designed especially for this latter use. It permits one to use the indexing system to step through the several fields of a record located at an arbitrary memory location.

In an early design stage, when we had a full-word format, the many address modification options were specified by individual bits. There was a bit in every order for indirect addressing and another for immediate addressing, etc. We found that performance would be improved by specifying the less commonly used features with separate half words. These are incorporated in the index arithmetic orders. Thus, indirect addressing is done by loading an index register with an indirect address, in a process which can be chained indefinitely. As a result, we were able to compress many of the common orders into a half-word format, while those that require more information use a full word. Full and half-word instructions may be intermixed freely.

Another index feature, renaming, permits one to use one index register and a set of memory words as if the combination were a set of index registers used one at a time. This is accomplished by having the machine remember which memory word is currently in the index register. Renaming causes a new memory word to be placed in the index register after its contents have been returned to their proper memory position. This gives one considerable facility in using sixteen index registers for a larger number of indexing quantities. The assumption here is that one will occasionally need more than sixteen index quantities, but one will rarely need more than sixteen quantities active at one time.

A third example of the indexing features is an order which stores an index value in the address field of an order. This seems unexceptional until one realizes that address fields are of different sizes, and the field size used by this operation is automatically determined by the operation code bits already in the destination location.

STRETCH includes a block transmission system which is more facile than any that has previously appeared. In addition to the usual operations, it provides for swapping, or interchanging, two blocks of data.

Branching is unusually simple and powerful. A single order permits one to test one of 64 automatically set machine status indicators, turn it off if desired, and branch upon either result of the test. Another conditional branching order permits one to perform a similar test upon any bit in memory. A third operation combines index incrementing and branching in a manner reminiscent of the 704 TIX order. These three orders constitute the complete conditional branching repertoire.

Almost all the internal registers of the machine which can be addressed by implication, such as the accumulator, can also be explicitly addressed by an ordinary memory address. Experience with the 650 has shown that the usefulness of this quite justifies the extra design effort and equipment it requires.

The input-output system is designed around an Exchange, which time-shares control circuitry among up to thirty-two simultaneously operating channels. These are designed so as to appear symmetric to the coder, regardless (almost) of the device attached. Thus, the same read order may cause a card, a tape, a console, or a disk file to enter its data into the memory. The input-output control word has been designed to match the index word format, so that a single programming technique can be used for control of the diverse operations of input, processing, and output.

Among the new i.-o. devices available with STRETCH, the disk file is of especial interest. Each such unit provides four million words of storage, divided into addressable arcs of 1024 words. The access time to an arc ranges from one to two tenths of a second. Words flow at the rate of one per four

microseconds.

6. Arithmetic and Logic

The two sets of data arithmetic instructions share several novel concepts. One of the most important is that of the universal accumulator, which contains augends and sums, multipliers and products, dividends and quotients. This constitutes a considerable convenience, and is a real advance over the accumulator - MQ concept which has been almost universal in earlier machines.

The treatment of signed quantities has been made flexible through the use of sign modifiers on all arithmetic instructions. These permit the sign of any operand to be inverted, treated as positive, or treated as negative.

Operands can also have non-numeric flag bits attached. These may be used to mark data for program control. Whenever a flagged operand is used, an indicator is set which may be allowed to cause immediate program interruption.

The floating point system is the primary arithmetic system of the computer. Its design has been based upon a careful study of the fundamental properties of floating point numbers. This has led to a unified treatment of the perennial problems of pretision and zero in semi-logarithmic number systems. As a result, the STRETCH floating point system provides (1) more precision in single precision operations than any other machine, (2) special facilities so that double and multiple precision operations become simple and straightforward, and (3) a unique noisy mode by which significance can be estimated so that one can tell when to use multiple precision.

In addition to these fundamental features of floating point, its operation set includes several niceties. Ordinary 704-style division is supplemented by reciprocal division, in which the accumulator contents are divided into those of the addressed location. An automatic square root operation and cumulative multiplication are provided. Exponents and fractions are so laid out in the format that they may be independently used as operands for variable field length arithmetic. Special floating point operations also operate on these fields.

The variable field length arithmetic system incorporates the ability to address memory with resolution to the bit and to operate upon memory fields than begin at any bit and may extend up to sixty-four bits, crossing a word boundary if desired. The basic character or digit structure of a field may be specified, and this so-called byte size may be from one to eight bits. Arithmetic may be performed in the binary or decimal radix, and operands may be stored with or without signs and flags. One useful feature of this fixed-point instruction set is a division operation which will perform any division with non-zero divisor. In addition to its obvious usefulness in commercial and control problems, variable field length arithmetic has great handiness for a wide variety of housekeeping operations which often constitute the most difficult part of scientific computations.

The three arithmetic units are supplemented by a separate logical unit. This is used in conjunction with the variable field length apparatus to provide an exceptionally powerful set of logical operations. These connective operations provide for combining memory bits and accumulator bits according to any of the sixteen logical functions of variables. The result may replace either operand or may be discarded. In any case, analyses are made of the results to determine if any result bit is one, how many result bits are one, and the address of the

leftmost one result bit. The byte size mechanism, provided for handling characters of different sizes in arithmetic, is utilized in connective operations to permit bit patterns to be interspersed, spread apart, or compressed.

Finally, the floating point hardware is used to provide automatic high-speed decimal-binary and binary-decimal conversion operations. These convert not only the radix of an operand, but also its format; thus, decimal variable field length operands are converted into binary floating point operands, and vice versa.

7. Conclusion

There are many other noteworthy, if minor, innovations in STRETCH. These, after all, are the subject matter of the next two weeks of the course, so I shall not attempt to cover them all here.

The combination of organizational innovations and sharp technological advances has led us to a computer whose power, flexibility, and generality approach the ambitious goals originally set. This has been possible because technological advances and planning advances have coincided, and because the freedom from historical constraints has allowed new concepts to be utilized fully. The result is a computer which is not a warm-over, not a mere amplification or enlargement of our earlier products. It is an instrument which provides unsurpassed power and convenience for larger problems, for more complex problems, for totally new kinds of problems. I am sure that it will stimulate startling new advances in programming techniques, in computer operation techniques, and in basic problem-solving techniques. In the next two weeks we shall enjoy introducing you to IBM's new contribution to mankind's tools. We know you are going to enjoy introducing it to our customers.